Step 6: test and validate your application

Beginner reference guide

Start with the working application from Step 5. Check what it does when users enter good data, make mistakes or encounter missing files. Record what you expected and what you actually saw.

What your team will submit

- A GitHub branch with your runnable app and relevant test code.
- A test report with expected and actual results, status, evidence, fixes and retests.
- A portal submission with your repository URL and the branch and evidence paths in the completion note.

How to use the SC11 example

- SC11 is the instructor's water-quality prediction example. Use your own project's inputs, rules, constraints and outputs when you plan and run tests.
- For a fuzzy controller, test rule coverage and limits. For an optimizer, check hard constraints and objective values. For a prediction app, compare the app with the saved method on the same inputs.

Read the SC11 examples as a worked demonstration. For each test, write what you will change, why you chose that case and what you expect. Then run it, record the actual result and apply the same idea to your project.

Deadline • 22 November 2026, 11:59 PM IST

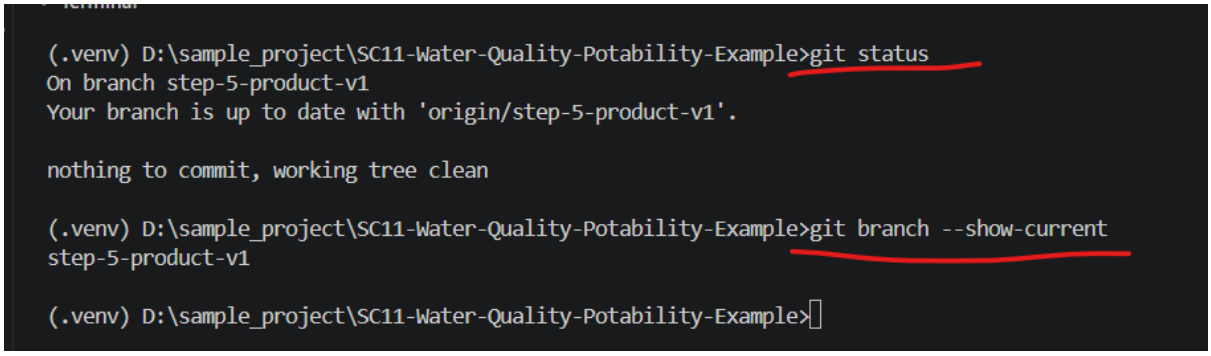
Action 1: start from your Step 5 branch

This gives you a clean record of what changed during testing.

- In VS Code, open your existing project folder. Save your work. If you only have the GitHub copy, choose Git: Clone from the Command Palette, paste your team's repository URL, then open the cloned folder.
- In the terminal, run git status. If you have unfinished changes, review and save them on the correct branch before switching.

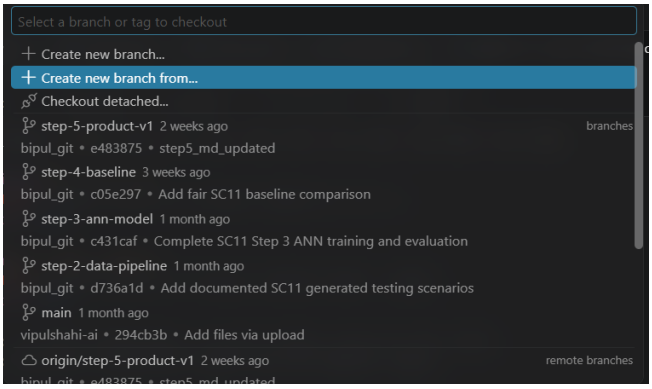
```
git status
git branch --show-current
```

What we expected. Git should show the completed Step 5 branch and a clean working tree before we branch off.



What happened. Git reported step-5-product-v1 and a clean working tree.

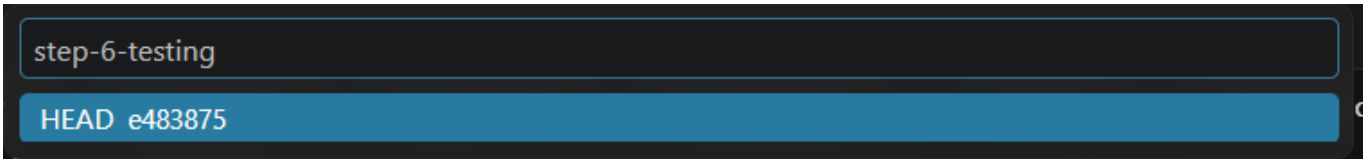
- Click the branch name at the bottom left. Choose Create new branch from..., select the branch with your submitted Step 5 app and name the new branch step-6-testing. SC11 starts from step-5-product-v1.



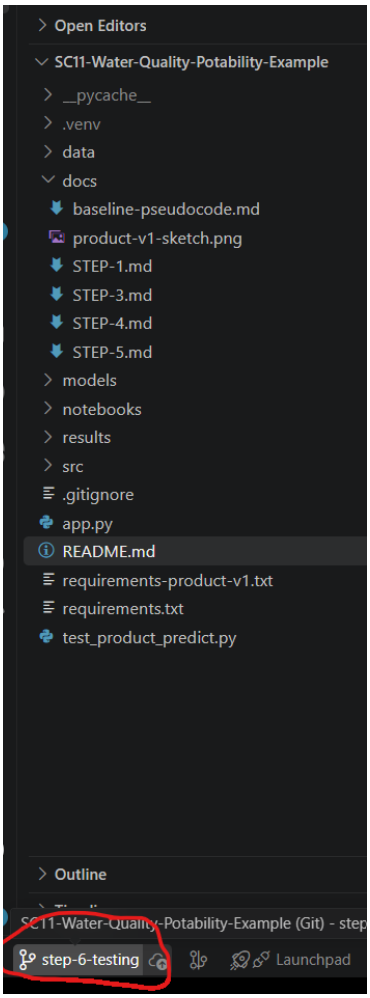
Choose Create new branch from... and select the completed Step 5 branch.

Action 1: name and confirm the new branch

- What we did. We entered step-6-testing and pressed Enter.
- What we expected. VS Code should switch to the new branch and keep the Step 5 files visible.



Enter the Step 6 branch name. HEAD is the selected starting commit in this example.



What happened. The Explorer showed the Step 5 app and the status bar showed step-6-testing.

In your project, confirm the new branch name and check that the Step 5 files you plan to test are present.

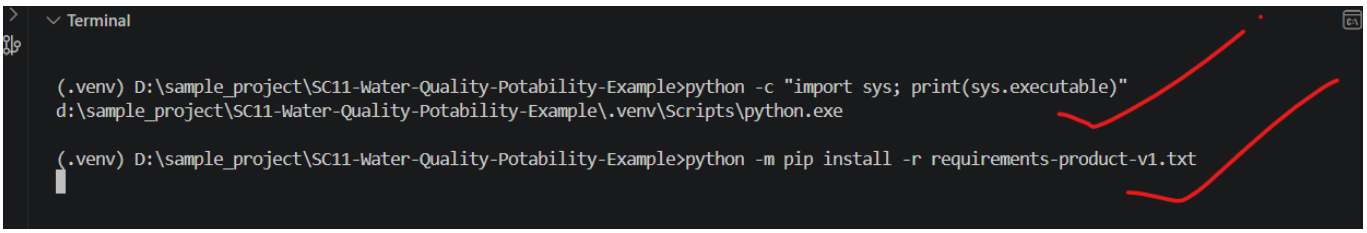
Action 2: launch the Step 5 app

A reviewer should be able to start the app by following your written instructions.

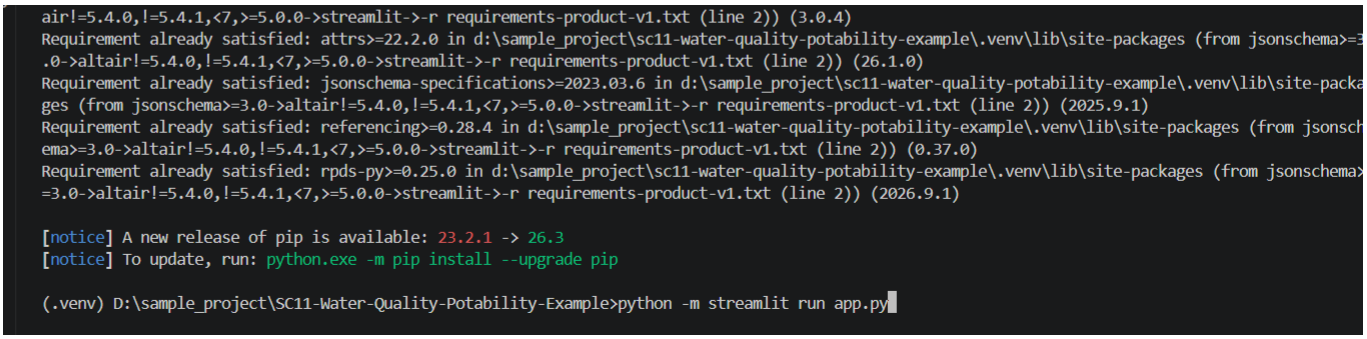
- What we did. We used the same Python environment as Step 5 and ran the app from the project folder.
- Open Terminal > New Terminal. In VS Code, use Python: Select Interpreter to choose the Step 5 environment. Open a new terminal and check the interpreter path. If needed, use the environment's Python executable for the commands below.
- Use your project's run command. For SC11, install the dependencies if needed, run Streamlit and open the local URL shown in the terminal. Keep that terminal open.

What we expected. The interpreter path should point to the project environment, and Streamlit should print a local address.

```
python -c "import sys; print(sys.executable)"
python -m pip install -r requirements-product-v1.txt
python -m streamlit run app.py
```



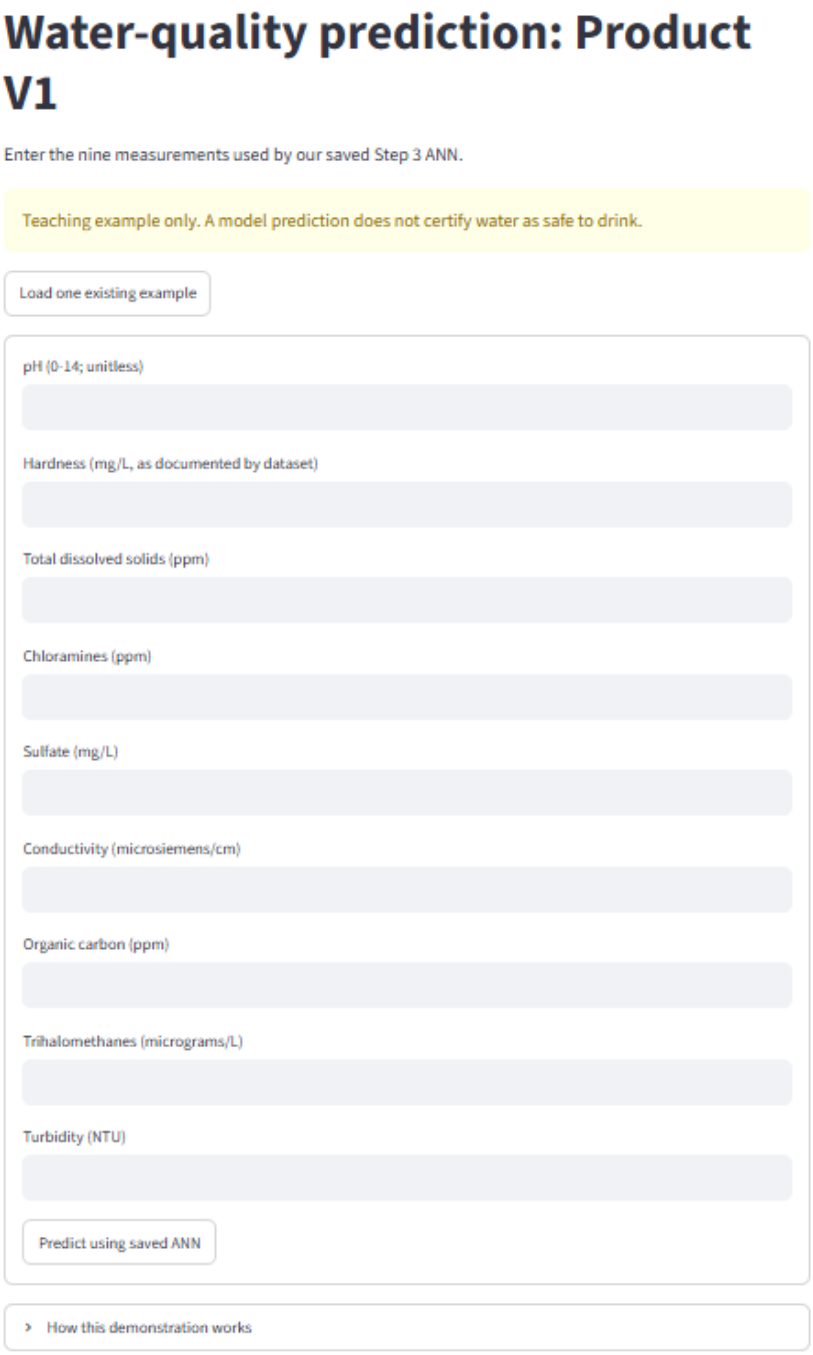
Check the .venv interpreter path, then install the Product V1 requirements if needed.



After the installation finishes, run the Streamlit command from the project root.

Action 2: check the running app

Open the local address shown in the terminal. The SC11 launch used http://localhost:8501.



What happened. The water-quality form opened at the local address. The sample is used in Action 4.

Start your own app using its launch command. Record the command and verify that the first screen opens.

Action 3: create the test plan

Write the expected behaviour before you run a test. Later you will compare it with what the application actually does.

What we did. We opened a test report and recorded which Git commit and Python version we were about to test.

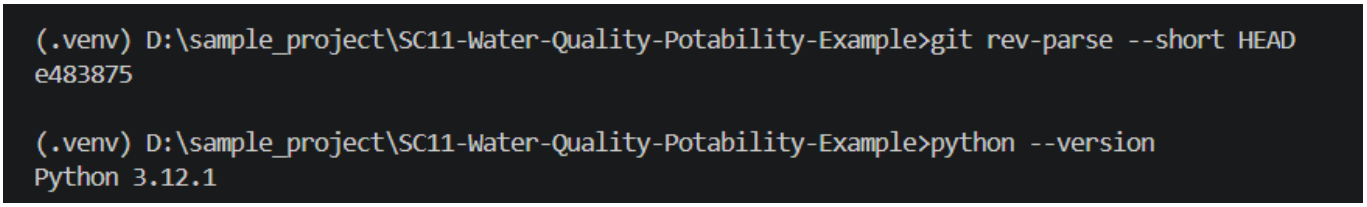
- 1. In VS Code, check that step-6-testing is shown at the bottom left. In Explorer, right-click docs, choose New File, and type STEP-6.md. Press Enter to open it.
- 2. Use the SC11 report on the following two pages to see how a test plan is organised. Reuse the headings and table columns. Write your own inputs, expected behaviour and pass conditions from your project's requirements.
- 3. Before starting Action 4, choose Terminal > New Terminal. Use a terminal at the project root, where app.py is located, with the Step 5 .venv active. Keep the original terminal running the app. Enter each command below and press Enter after it.

```
git rev-parse --short HEAD
python --version
```

The first command prints the short ID of the current Git commit. It identifies the version of the app you are about to test. The second command prints the Python interpreter version used in that terminal.

What happened in SC11

```
e483875
Python 3.12.1
```



The first output is the commit ID. The second output starts with the word Python.

Action 3: full Markdown template, part 1

Read this SC11 example to see how the team, environment and test cases fit together. Long source lines wrap on the page. The attached Markdown preserves the original formatting.

```
# Step 6 - Testing and validation

## Project and environment

- Project: SC11 Water Quality Potability Example (instructor reference).
- Branch: `step-6-testing`.
- Commit tested: [run `git rev-parse --short HEAD` and enter the result].
- Python and dependencies: [record the versions actually used].
- Launch from the repository root: `python -m streamlit run app.py`.
- Reference input: first data row in `data/processed/validation.csv`.

## Planned tests and results

Keep the other eight fields fixed when testing a single input. For each run, replace the
    blank observation with what the interface or command actually showed. Use PASS, FAIL
    or NOT RUN. Record a relative evidence path or a short exact output.

| ID | Input or action | Expected behaviour | Actual result | Status |
| --- | --- | --- | --- | --- |
| T01 | Load existing sample and predict | Display a class, label and educational notice.
    Allow CSV download. | Pending | NOT RUN |
| T02 | Reload sample and set pH to 0 | Accept the lower limit and display a prediction.
    | Pending | NOT RUN |
| T03 | Reload sample and set pH to 14 | Accept the upper limit and display a prediction.
    | Pending | NOT RUN |
| T04 | Reload sample and set pH to 15 | Show the pH range message and withhold the
    result. | Pending | NOT RUN |
| T05 | Reload sample and enter abc as pH | Ask for a number and withhold the result. |
    Pending | NOT RUN |
| T06 | Reload sample and clear Trihalomethanes | Ask for a value and withhold the
    result. | Pending | NOT RUN |
| T07 | Start a disposable copy with the model file renamed | Explain that the saved
    model or metadata is missing and stop before prediction. | Pending | NOT RUN |
| T08 | Restore the copied model and relaunch | The normal sample predicts again. |
    Pending | NOT RUN |
| T09 | Compare normal-case UI export with the reference CSV | Nine inputs, predicted
    class and label agree. | Pending | NOT RUN |
| T10 | Run `python -m unittest -v test_product_predict` | Five validation methods run
    and finish with OK. | Pending | NOT RUN |
```

Action 3: full Markdown template, part 2

Continue in the same file below the test table. Leave the evidence and retest prompts ready for the later actions.

```
## Evidence

| Test ID | File path or exact observed output |
| --- | --- |
| [ID] | [path or observed output] |

## Failures, fixes and retests

| Test ID | What failed | What changed and where | Result of repeating the case and a
    normal case |
| --- | --- | --- | --- |
| [ID] | [observed failure] | [file and change] | [observed result] |

If you find no failure, say which checks passed and record any remaining limits here.

## Known limits

- This app demonstrates a saved model. A match between the app and script verifies
    consistency on the tested sample.
- The app cannot certify drinking-water safety.
- [State the data, devices, cases or environments you did not test.]

## Contributions

- Instructor example: [record your work].
- Student teams: list each member and the tests, fixes or evidence they completed.
```

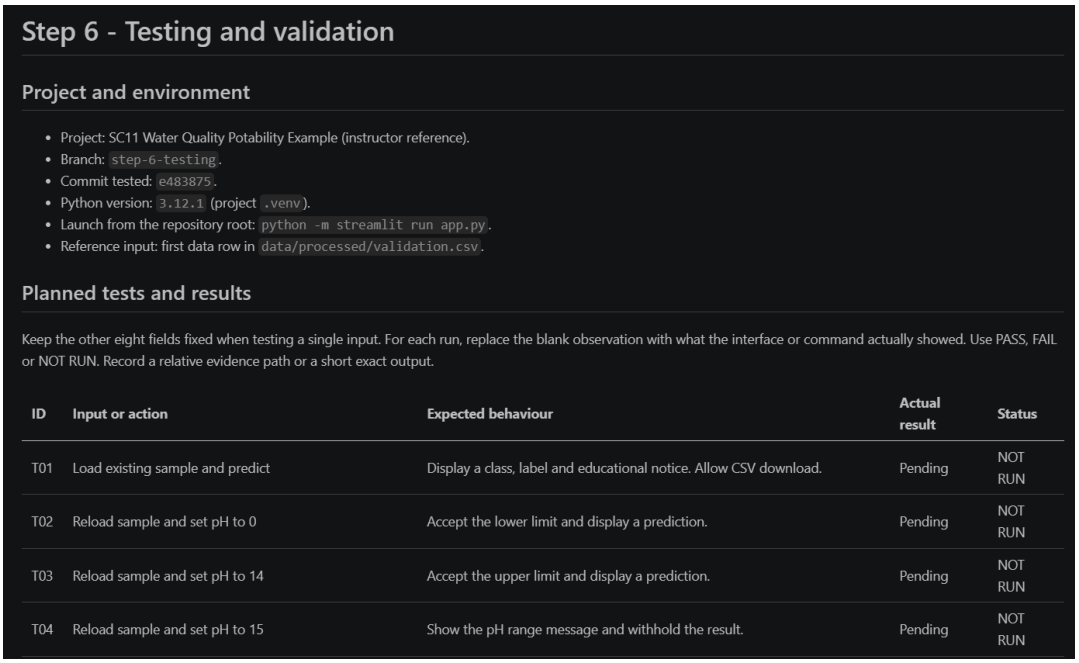
Action 3: fill in, save and preview the plan

What we expected. The preview should show the environment details and test table. Results stay pending until each check runs.

- 4. In your report's Project and environment section, enter your branch, commit and Python version. The lines below show how SC11 recorded its results. Keep just the version number from the Python output.

```
- Commit tested: `e483875`.
- Python version: `3.12.1` (project `.venv`).
```

- 5. Leave Actual result as Pending and Status as NOT RUN for each test in your plan. Fill these columns after observing the result of each test.
- 6. Click in the STEP-6.md editor and press Ctrl+S. This writes your edits to the file on disk. In VS Code, the unsaved dot on the tab disappears after the save.
- 7. With STEP-6.md active, press Ctrl+Shift+V. This opens Markdown Preview. Check that the headings and table render and that the plan shows what you expect to test. Make corrections in the Markdown editor and save again.



Step 6 - Testing and validation

Project and environment

- Project: SC11 Water Quality Potability Example (instructor reference).
- Branch: `step-6-testing`.
- Commit tested: `e483875`.
- Python version: `3.12.1` (project `.venv`).
- Launch from the repository root: `python -m streamlit run app.py`.
- Reference input: first data row in `data/processed/validation.csv`.

Planned tests and results

Keep the other eight fields fixed when testing a single input. For each run, replace the blank observation with what the interface or command actually showed. Use PASS, FAIL or NOT RUN. Record a relative evidence path or a short exact output.

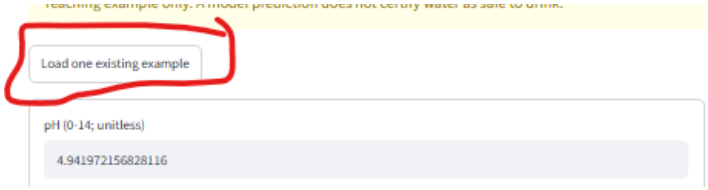
ID	Input or action	Expected behaviour	Actual result	Status
T01	Load existing sample and predict	Display a class, label and educational notice. Allow CSV download.	Pending	NOT RUN
T02	Reload sample and set pH to 0	Accept the lower limit and display a prediction.	Pending	NOT RUN
T03	Reload sample and set pH to 14	Accept the upper limit and display a prediction.	Pending	NOT RUN
T04	Reload sample and set pH to 15	Show the pH range message and withhold the result.	Pending	NOT RUN

What happened. The preview showed the SC11 commit and Python version, the expected results and the test rows marked NOT RUN.

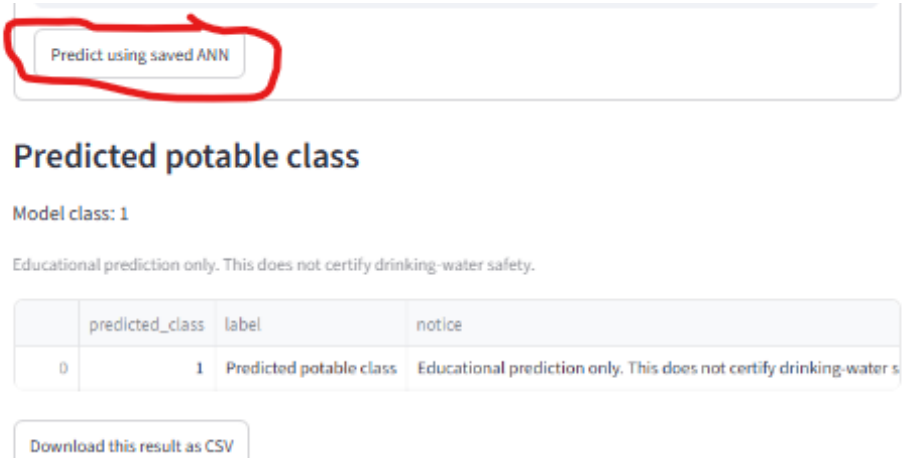
Action 4: run the normal prediction (T01)

Choose a typical valid input for your project. Check whether a user can get a clear result and save it as evidence. SC11 demonstrates this with a water-quality sample.

- 1. Open the running app at localhost:8501. Click Load one existing example. This fills the form with nine measurements from the first row of the validation CSV.
- 2. Scroll down and click Predict using saved ANN. We expected the app to display a class, a readable label and the educational-use notice.
- 3. Click Download this result as CSV. The next page shows how to locate the file, whatever folder your browser uses.



Load the sample to fill the form.



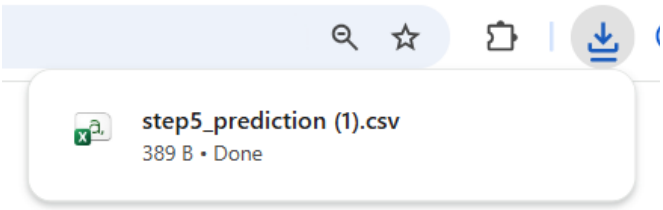
What happened. SC11 displayed class 1, Predicted potable class, and offered a CSV download.

In your project, record the input, the result you observed and how you saved the evidence.

Action 4: find and save the browser CSV

Keep a copy of the app output in your project so a teammate can find the exact evidence used for comparison.

- 4. In Chrome on Windows, open Downloads with Ctrl+J. Find your latest CSV by its name and time. Choose Show in folder beside it, or use the folder icon in the download tray. File Explorer opens the folder where the browser actually saved it.
- Chrome normally uses your Windows Downloads folder. If the file went somewhere else, use Show in folder or check the Location under Chrome Settings > Downloads. If the browser opens a Save As dialog, choose your project evidence folder directly.



SC11's latest file is step5_prediction (1).csv. A numbered suffix can appear when a file with the original name already exists.

- 5. In File Explorer, select the latest file and press Ctrl+C. Open the project and navigate to data > reports, then press Ctrl+V. Press F2 to rename the copy. SC11 uses step6_ui_prediction.csv. In Windows 11, View > Show > File name extensions helps you check that the name ends in .csv once.
- If you already saved it in your project folder using Save As, check its name and path there. For your own project, choose a clear evidence folder and record the relative path in your test report. Use the same path in your script.

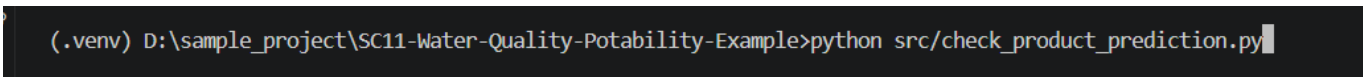
Chrome download help: support.google.com/chrome/answer/95759

Action 4: generate a reference result for T09

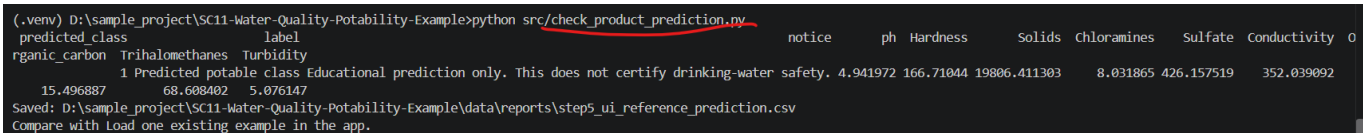
Compare your app's result with a reference generated for the same input and configuration. This checks whether the app passes the inputs to the method and displays the result consistently.

- 6. Use your project's existing model, controller or solver to produce the reference. SC11 already has a script that loads the first row of the validation CSV, runs the same saved model and writes the reference CSV.
- For the SC11 walkthrough, leave the app running. In VS Code choose Terminal > New Terminal, check that you are in the folder containing app.py, and run this command with the project .venv active.

```
python src/check_product_prediction.py
```



Run the existing script from the project root.



The script prints class 1 and the saved path for step5_ui_reference_prediction.csv.

The reports folder now contains the reference file from Python and the copy of the browser download.

```
data/reports/step5_ui_reference_prediction.csv
data/reports/step6_ui_prediction.csv
```

predicted_	label	notice	ph
1	Predicted potable class	Educational prediction only. This does not certify drinking-water safety.	4.94

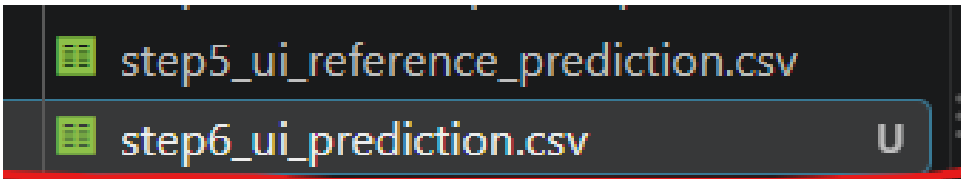
The downloaded row contains the class, label, notice and measurements. This view shows the first measurement column, pH.

Action 4: design your comparison script

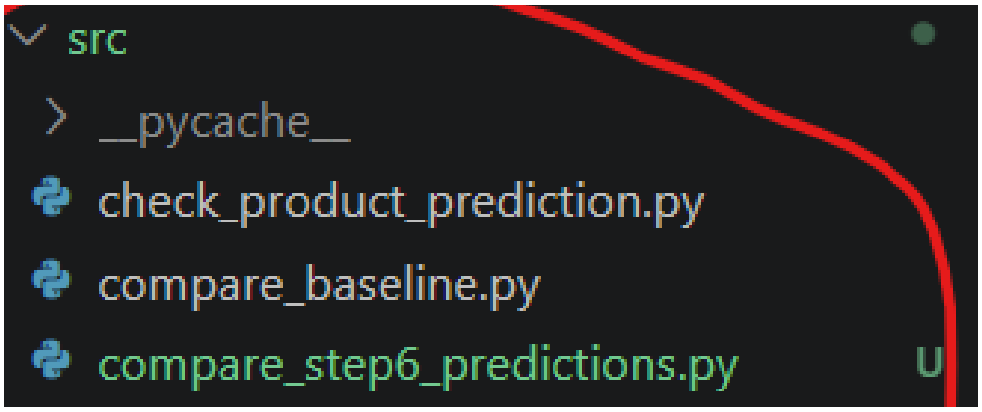
Write a script for your own project. The SC11 code on the next two pages shows how to read CSVs, compare fields and report differences.

- 7. Decide what to compare. Identify the same test input in both results. Check exact values for labels and decisions. For decimal values, choose and document a tolerance that makes sense for your units. For example, an irrigation team could compare soil moisture and the watering recommendation.
- 8. In VS Code Explorer, right-click the folder where your project keeps its scripts, choose New File, and name your Python file. Read the two saved results, check that the required fields are present, compare them, and print PASS or list the differences. Save with Ctrl+S.
- SC11's example layout is shown below. Use your team's filenames and input and output fields when writing your script.

```
src/check_product_prediction.py
src/compare_step6_predictions.py
data/reports/step5_ui_reference_prediction.csv
data/reports/step6_ui_prediction.csv
```



Both CSV files are visible under data > reports in the SC11 project.



The two Python scripts are visible together under src.

Action 4: SC11 comparison code, part 1

SC11 reads one result row per CSV and checks that every required field has a value. Use your own file paths, fields and record matching rules.

```
"""Compare the SC11 reference CSV with the normal-case app download."""
import argparse
import csv
from math import isclose
from pathlib import Path

from product_predict import FEATURES, ROOT


REPORT_DIR = ROOT / "data" / "reports"


def read_prediction(path):
    with path.open(encoding="utf-8-sig", newline="") as source:
        rows = list(csv.DictReader(source))
    if len(rows) != 1:
        raise ValueError(f"{path.name}: expected exactly one prediction row.")
    row = rows[0]
    for field in [*FEATURES, "predicted_class", "label"]:
        if not row.get(field):
            raise ValueError(f"{path.name}: missing value for {field}.")
    return row
```

Action 4: SC11 comparison code, part 2

SC11 compares measurements within a tolerance and matches the class and label exactly. Choose comparison rules that fit your project's outputs.

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "reference", type=Path, nargs="?",
        default=REPORT_DIR / "step5_ui_reference_prediction.csv",
    )
    parser.add_argument(
        "app_download", type=Path, nargs="?",
        default=REPORT_DIR / "step6_ui_prediction.csv",
    )
    args = parser.parse_args()

    try:
        reference = read_prediction(args.reference)
        app = read_prediction(args.app_download)
        differences = [
            name for name in FEATURES
            if not isclose(float(reference[name]), float(app[name]),
                           rel_tol=1e-9, abs_tol=1e-9)
        ]
        differences.extend(
            name for name in ("predicted_class", "label")
            if reference[name].strip() != app[name].strip()
        )
    except (OSError, ValueError, csv.Error) as error:
        parser.exit(1, f"FAIL: {error}\n")

    if differences:
        parser.exit(1, f"FAIL: Different values in {', '.join(differences)}.\n")
    print("PASS: All nine inputs, the predicted class and the label match.")
    print(f"Class {app['predicted_class']} - {app['label']}")

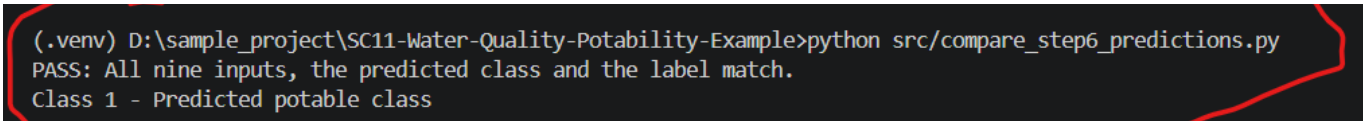

if __name__ == "__main__":
    main()
```

Action 4: run the comparison and record T09

Run the script you wrote and record what it found. In SC11 the checks cover nine measurements, the predicted class and the displayed label.

- 9. In a terminal at your project root, run your comparison script using its actual path. The command below shows what we ran in SC11.
- What we expected. Both results should contain the same nine measurements, predicted class and displayed label.

```
python src/compare_step6_predictions.py
```



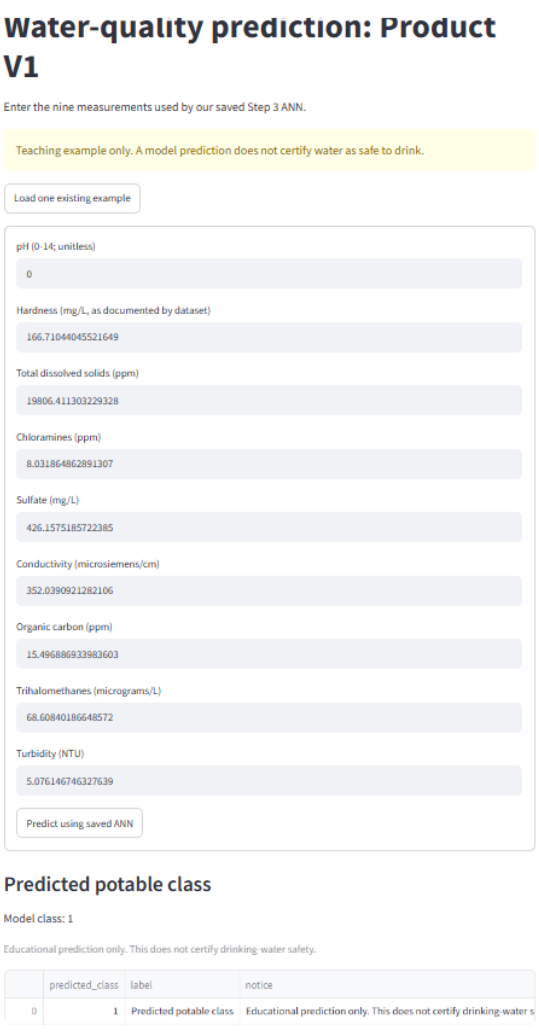
What happened. The SC11 comparison printed PASS. Both files contained class 1 and the label Predicted potable class.

- 10. Record your observed result, status and both evidence paths in the report. In SC11's T09, all nine inputs matched and both files showed class 1, Predicted potable class. That matched the expected behaviour, so T09 passed.
- If Python says 'can't open file', check the terminal folder and the script path. Check that the filename ends in .py. If the script reports a missing CSV, check where each result was saved and the path in your script.
- If the script reports a difference, check the input values and configuration on both sides. Then repeat the test and record what changed and what you observed.

Action 5: test the lower and upper limits

A boundary test checks an exact limit in your project requirements. It can catch a rule that rejects a value your app should accept.

- Choose a field or constraint with a defined limit. SC11 accepts pH from 0 through 14, including both ends. Other projects may use limits on load, moisture or the last timetable slot.
- What we did. We loaded the sample, set pH to 0 and predicted for T02. We then reloaded the sample, set pH to 14 and predicted for T03. The other eight measurements stayed the same.
- What we expected. Both boundary inputs should be accepted and return a prediction.



What happened in T02. The app accepted pH 0 and displayed class 1.



What happened in T03. The app accepted pH 14 and displayed class 0.

Action 5: record what each boundary test showed

Both SC11 checks passed because the app accepted the boundary and gave a prediction in each case. The predicted class is part of the observation.

- In your project, test each documented boundary separately. Record the exact input, the observed output and whether the app handled the boundary as expected.
- You can also check a value just inside a limit. The extra SC11 example below uses pH 1. It was accepted and returned class 1. This provides another observation close to the lower limit.



Extra check. pH 1 is inside the accepted range and the app displayed class 1.

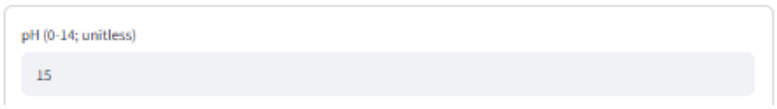
Action 6: change pH to 15 (T04)

SC11 accepts pH from 0 through 14. We enter 15 to check that the app rejects a number just above the upper limit and explains the valid range.

- What we did. We loaded the SC11 sample, changed only pH to 15 and clicked Predict using saved ANN.
- What we expected. The app should show the allowed range and withhold the prediction.



SC11 form after submitting the test input.



Field under test.



What happened. The app displayed 'ph: use a value from 0 to 14.' and withheld the prediction. T04 passed.

In your project, choose a numeric input with a documented limit. Try one value beyond it and record the message and whether the app produced an output.

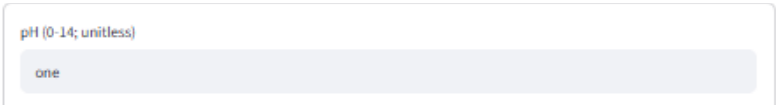
Action 6: enter text in the pH field (T05)

The pH field requires a number. We used the word one to check whether the app explains the input type when a user enters text.

- What we did. We loaded the sample again, replaced pH with one and submitted the form. The test plan used abc as another example of text.
- What we expected. The app should ask for a number and withhold the prediction.



SC11 form after submitting the test input.



Field under test.



What happened. The app displayed 'ph: enter a number.' and withheld the prediction. T05 passed for the word one.

In your project, try text in a numeric field. If your form prevents typing text, record what it did and whether it helped the user correct the input.

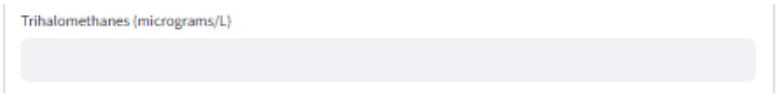
Action 6: leave a required field blank (T06)

SC11 needs all nine measurements. We cleared Trihalomethanes to check that the app names the missing field.

- What we did. We used valid values in the other eight fields, including pH 14, cleared Trihalomethanes and submitted the form.
- What we expected. The app should ask for the missing measurement and withhold the prediction.



SC11 form after submitting the test input.



Field under test.

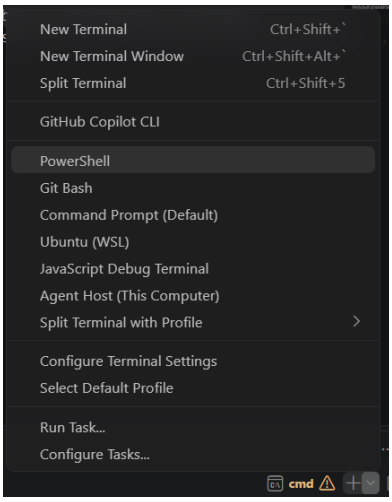
What happened. The app displayed 'Trihalomethanes: enter a value.' and withheld the prediction. T06 passed.

In your project, clear one required field and leave the other inputs valid. Record the expected behaviour, the actual message and output, and PASS or FAIL for each invalid-input case.

Action 7: prepare a separate copy for T07 and T08

SC11 needs a saved model before it can predict. We test in a temporary copy so the original project and its model stay intact.

In VS Code, open the Terminal panel. Click the arrow beside the + button and choose PowerShell. Start in the SC11 project folder containing app.py.



Open a second terminal using the PowerShell profile.

Run these lines one at a time in the new terminal. The first saves the path to Python from the original project. The others make the copy and switch the terminal into it.

```
$python = (Resolve-Path ".venv/Scripts/python.exe").Path
$copy = Join-Path $env:TEMP ("sc11-step6-" + [guid]::NewGuid())
New-Item -ItemType Directory -Path $copy | Out-Null
Copy-Item -Path app.py,src,data,models -Destination $copy -Recurse
Set-Location -LiteralPath $copy
```

```
(.venv) PS D:\sample_project\SC11-Water-Quality-Potability-Example> $python = (Resolve-Path ".venv/Scripts/python.exe").Path
(.venv) PS D:\sample_project\SC11-Water-Quality-Potability-Example> $copy = Join-Path $env:TEMP ("sc11-step6-" + [guid]::NewGuid())
(.venv) PS D:\sample_project\SC11-Water-Quality-Potability-Example> New-Item -ItemType Directory -Path $copy | Out-Null
(.venv) PS D:\sample_project\SC11-Water-Quality-Potability-Example> Copy-Item -Path app.py,src,data,models -Destination $copy -Recurse
(.venv) PS D:\sample_project\SC11-Water-Quality-Potability-Example> Set-Location -LiteralPath $copy
(.venv) PS C:\Users\vipul\AppData\Local\Temp\sc11-step6-eb015e81-c3fb-4d3d-bfc4-5ddf27fb7ab0>
```

Check the final prompt: it ends in a temporary sc11-step6 folder with a unique suffix. Yours will have a different suffix.

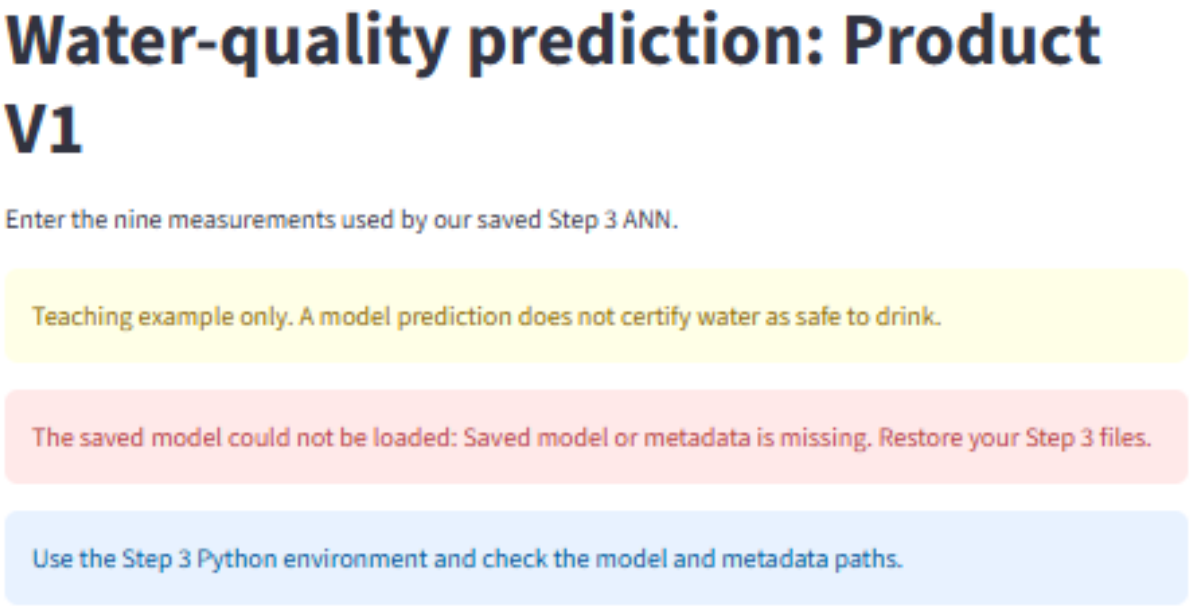
Action 7: test with the model file missing (T07)

We make the saved model temporarily unavailable to check that the app explains the problem before a user tries to predict.

Check that the PowerShell prompt is in the temporary copy from the previous page. Run these two lines there.

```
Rename-Item "models/ann_water_potability.pkl" "ann_water_potability.pkl.disabled"
& $python -m streamlit run app.py --server.port 8502
```

- What we did. Renaming the copy's model hides it from the app without deleting it. Port 8502 lets the copy run separately from the original app on port 8501.
- What we expected. Open http://localhost:8502. The copy should show a message about the missing model or metadata and hide the prediction form.
- What happened. The browser showed the missing-file message, told us to restore the Step 3 files and did not show the prediction form. Record that outcome as T07.
- In your project, pick a file your app needs to run, such as a saved model, rules file or timetable. Use a copy and check that the error tells a user what to do next.



T07 on port 8502: the red message says the saved model or metadata is missing. The prediction form is unavailable.

Action 7: restore the model and retry (T08)

Restoring the file and repeating the normal case checks whether the app recovers when its dependency is available again.

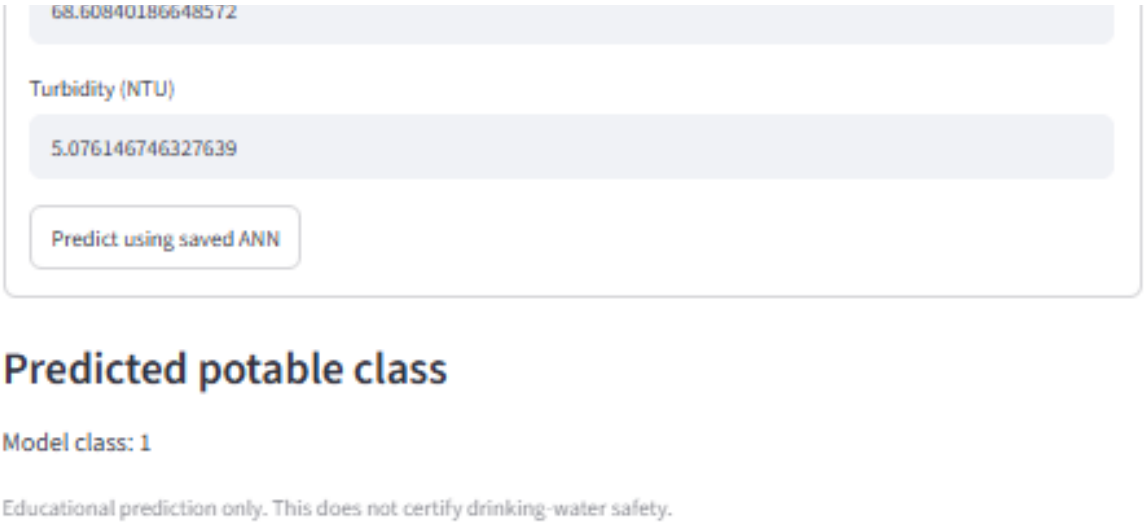
Press Ctrl+C in the second terminal to stop the test copy. Stay in the same terminal so it still has the saved Python path and is in the temporary folder. Run these lines to restore the original filename and restart the copy.

```
Rename-Item "models/ann_water_potability.pkl.disabled" "ann_water_potability.pkl"
& $python -m streamlit run app.py --server.port 8502
```

- What we expected. Reopen http://localhost:8502, click Load one existing example, then click Predict using saved ANN. The prediction should return for the same sample.
- What happened. The form returned and the browser showed Predicted potable class and Model class: 1. Record that as T08, then stop the test copy with Ctrl+C.
- In your project, restore the file and retry the same valid input. Record missing-file and recovery results in separate rows so a reviewer can check both.



Use the Local URL printed in the terminal.



T08 in the browser: the lower part of the restored form, its Predict button and the class 1 result.

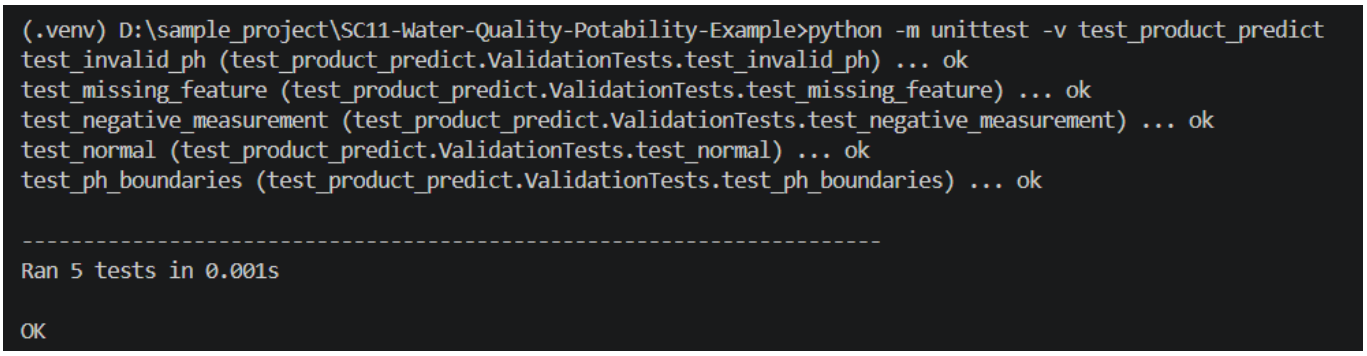
Action 8: run the automated checks (T10)

A unit test checks a small part of the code. The same tests can be run again after each change.

What we did. In a terminal at the original SC11 project root, we ran the existing test_product_predict.py file through Python's unittest runner. The -v option shows each test by name.

```
python -m unittest -v test_product_predict
```

- What we expected. Each test should show ok. If a check fails, the terminal identifies the test and shows the error or failed assertion.
- What happened. All five tests showed ok, followed by Ran 5 tests and OK. These checks exercise the input validator. The browser tests show how the app behaves when someone uses it.
- In your project, use the examples on the next page to write checks for your own rules. Record the command, test count and result alongside your browser evidence.



T10: the command, five named checks (invalid pH, missing field, negative measurement, normal input and pH boundaries), and the final OK.

Action 8: design a check for your own rules

SC11 uses assertEqual when it expects a value and assertRaises when it expects the validator to reject an input.

```
def test_normal(self):
    self.assertEqual(validate_inputs(self.values)['ph'], 7.0)

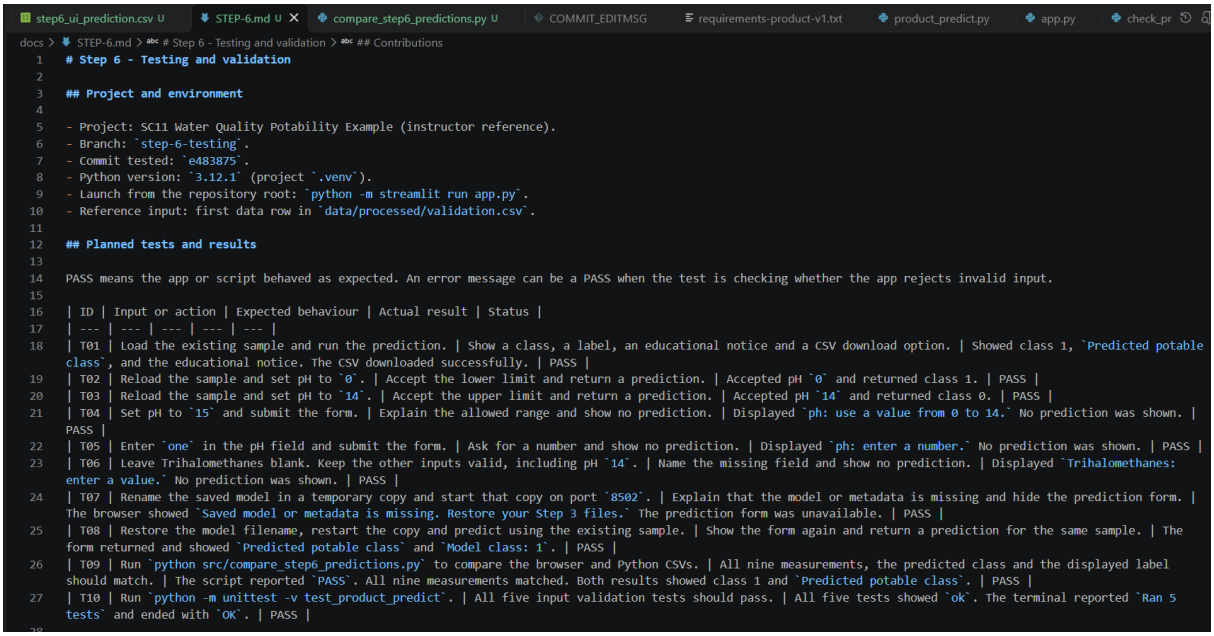
def test_out_of_range(self):
    self.values['ph'] = '15'
    with self.assertRaises(ValueError):
        validate_inputs(self.values)
```

- What the code does. These two examples assume setUp has filled self.values with valid measurements and pH 7. The first test expects a numeric 7.0. The second changes pH to 15 and expects a ValueError.
- Why we chose them. The checks exercise two outcomes in the validator. It should accept and convert a valid value and reject an out-of-range value.
- In your project, identify the function you want to test, choose a valid input and an invalid input, and write assertions for the behaviours your code should have. Run your test file and record what actually happens.

Action 9: update the Markdown file

Update the same docs/STEP-6.md file you created in Action 3. The Markdown editor is where you change text; Preview shows how the saved report will look.

- 1. In VS Code Explorer, open docs > STEP-6.md. Under Planned tests and results, fill Actual result and Status for each case you ran. Leave a test you did not run as NOT RUN.
- 2. Keep the vertical bars that separate the table cells. When you add a result, change Pending to what happened and NOT RUN to the correct status. Under Evidence, keep a file path from the repo or the exact result you observed.



```
step6_ui_prediction.csv U | STEP-6.md U | compare_step6_predictions.py U | COMMIT_EDITMSG | requirements-product-v1.txt | product_predict.py | app.py | check_pr
docs > STEP-6.md > # Step 6 - Testing and validation > ## Contributions
# Step 6 - Testing and validation

## Project and environment

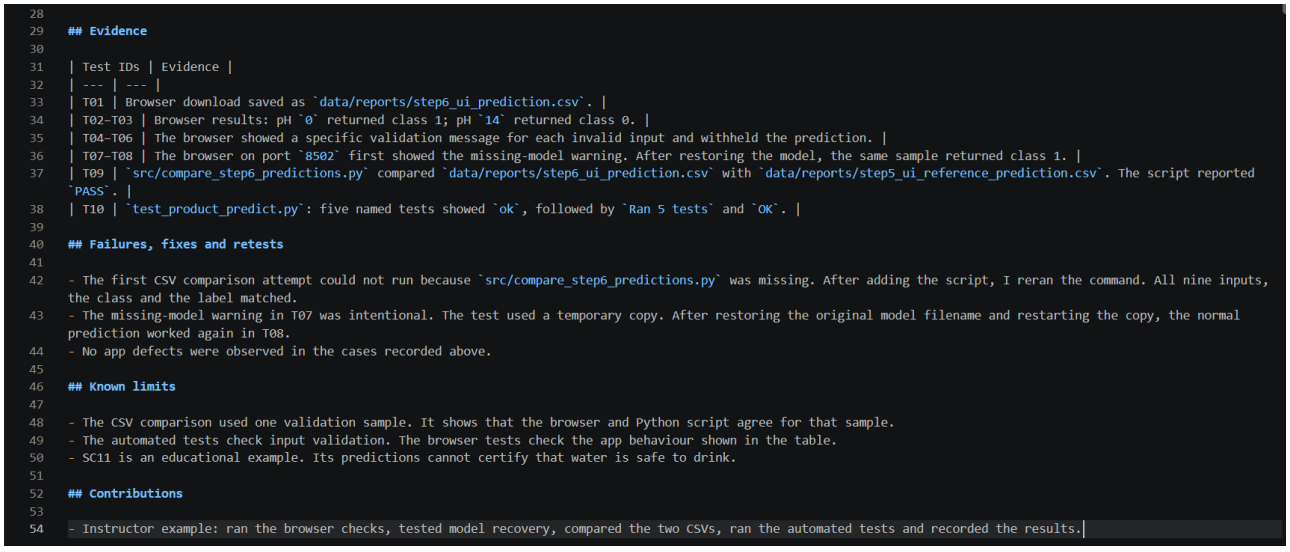
- Project: SC11 Water Quality Potability Example (instructor reference).
- Branch: `step-6-testing`.
- Commit tested: `e483875`.
- Python version: `3.12.1` (project `.venv`).
- Launch from the repository root: `python -m streamlit run app.py`.
- Reference input: first data row in `data/processed/validation.csv`.

## Planned tests and results

PASS means the app or script behaved as expected. An error message can be a PASS when the test is checking whether the app rejects invalid input.

| ID | Input or action | Expected behaviour | Actual result | Status |
| --- | --- | --- | --- | --- |
| T01 | Load the existing sample and run the prediction. | Show a class, a label, an educational notice and a CSV download option. | Showed class 1, `Predicted potable class`, and the educational notice. The CSV downloaded successfully. | PASS |
| T02 | Reload the sample and set pH to `0`. | Accept the lower limit and return a prediction. | Accepted pH `0` and returned class 1. | PASS |
| T03 | Reload the sample and set pH to `14`. | Accept the upper limit and return a prediction. | Accepted pH `14` and returned class 0. | PASS |
| T04 | Set pH to `15` and submit the form. | Explain the allowed range and show no prediction. | Displayed `ph: use a value from 0 to 14.` No prediction was shown. | PASS |
| T05 | Enter `one` in the pH field and submit the form. | Ask for a number and show no prediction. | Displayed `ph: enter a number.` No prediction was shown. | PASS |
| T06 | Leave Trihalomethanes blank. Keep the other inputs valid, including pH `14`. | Name the missing field and show no prediction. | Displayed `Trihalomethanes: enter a value.` No prediction was shown. | PASS |
| T07 | Rename the saved model in a temporary copy and start that copy on port `8502`. | Explain that the model or metadata is missing and hide the prediction form. | The browser showed `Saved model or metadata is missing. Restore your Step 3 files.` The prediction form was unavailable. | PASS |
| T08 | Restore the model filename, restart the copy and predict using the existing sample. | Show the form again and return a prediction for the same sample. | The form returned and showed `Predicted potable class` and `Model class: 1`. | PASS |
| T09 | Run `python src/compare_step6_predictions.py` to compare the browser and Python CSVs. | All nine measurements, the predicted class and the displayed label should match. | The script reported `PASS`. All nine measurements matched. Both results showed class 1 and `Predicted potable class`. | PASS |
| T10 | Run `python -m unittest -v test_product_predict`. | All five input validation tests should pass. | All five tests showed `ok`. The terminal reported `Ran 5 tests` and ended with `OK`. | PASS |
```

In the Markdown editor: the filled environment details and the test rows from T01 to T10.



```
## Evidence

| Test IDs | Evidence |
| --- | --- |
| T01 | Browser download saved as `data/reports/step6_ui_prediction.csv`. |
| T02–T03 | Browser results: pH `0` returned class 1; pH `14` returned class 0. |
| T04–T06 | The browser showed a specific validation message for each invalid input and withheld the prediction. |
| T07–T08 | The browser on port `8502` first showed the missing-model warning. After restoring the model, the same sample returned class 1. |
| T09 | `src/compare_step6_predictions.py` compared `data/reports/step6_ui_prediction.csv` with `data/reports/step5_ui_reference_prediction.csv`. The script reported `PASS`. |
| T10 | `test_product_predict.py`: five named tests showed `ok`, followed by `Ran 5 tests` and `OK`. |

## Failures, fixes and retests

- The first CSV comparison attempt could not run because `src/compare_step6_predictions.py` was missing. After adding the script, I reran the command. All nine inputs, the class and the label matched.
- The missing-model warning in T07 was intentional. The test used a temporary copy. After restoring the original model filename and restarting the copy, the normal prediction worked again in T08.
- No app defects were observed in the cases recorded above.

## Known limits

- The CSV comparison used one validation sample. It shows that the browser and Python script agree for that sample.
- The automated tests check input validation. The browser tests check the app behaviour shown in the table.
- SC11 is an educational example. Its predictions cannot certify that water is safe to drink.

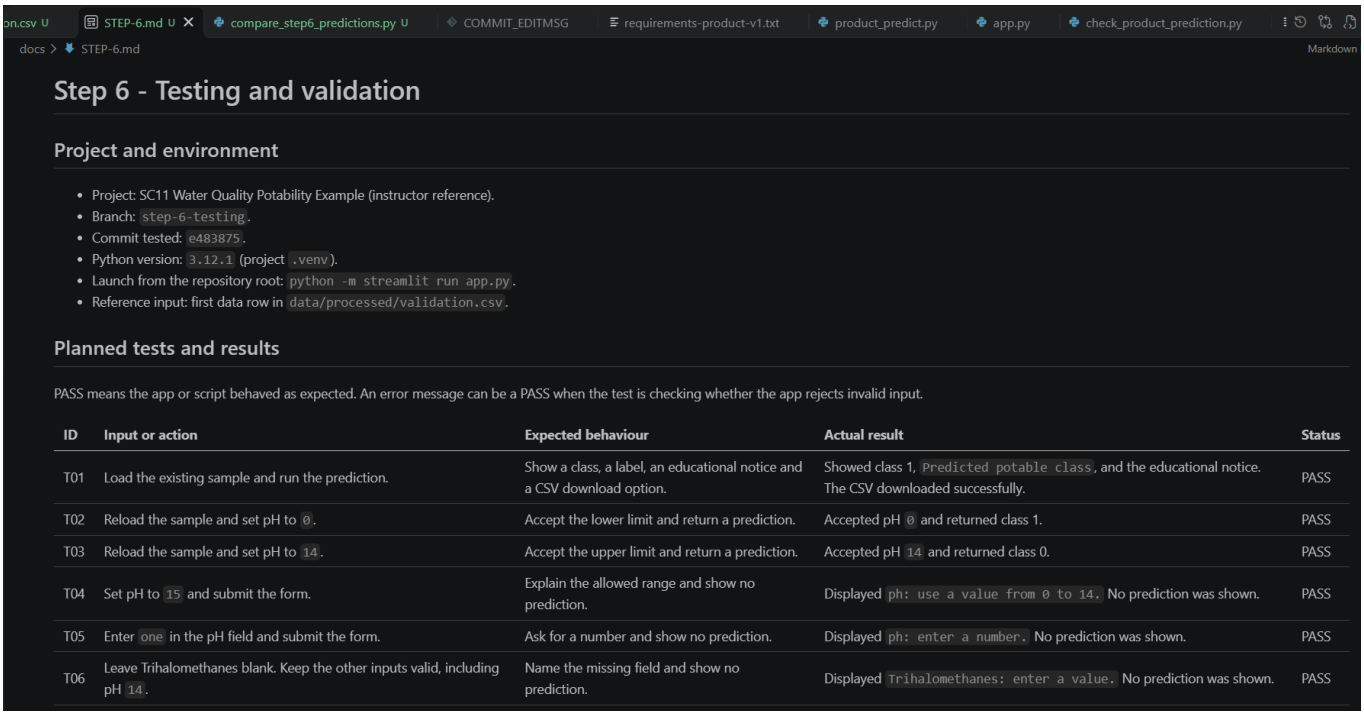
## Contributions

- Instructor example: ran the browser checks, tested model recovery, compared the two CSVs, ran the automated tests and recorded the results.
```

Scroll down in the same file to add evidence, fixes or retests, limits and contributions.

Action 9: check both halves of the results table

3. Press Ctrl+S to save and Ctrl+Shift+V to open Markdown Preview. Check the project details first, then scroll through the entire table.



on.csv U | STEP-6.md U | compare_step6_predictions.py U | COMMIT_EDITMSG | requirements-product-v1.txt | product_predict.py | app.py | check_product_prediction.py

docs > STEP-6.md — Markdown

Step 6 - Testing and validation

Project and environment

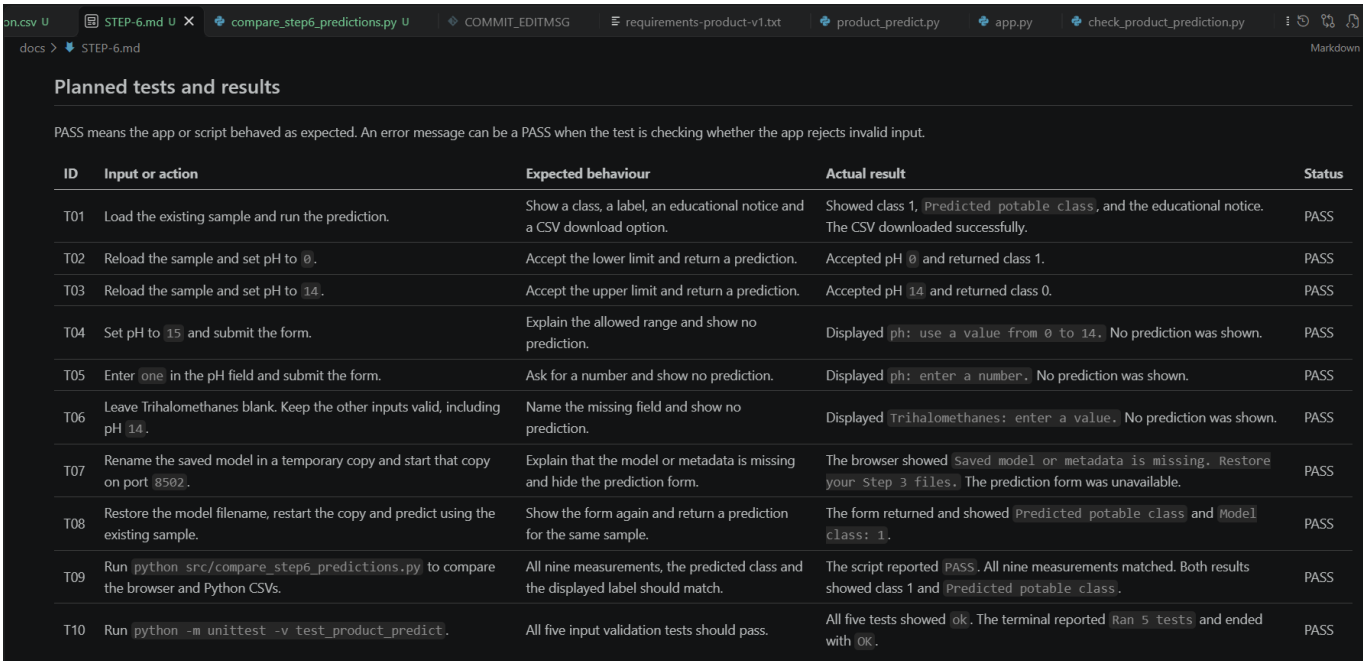
- Project: SC11 Water Quality Potability Example (instructor reference).
- Branch: `step-6-testing`.
- Commit tested: `e483875`.
- Python version: `3.12.1` (project `.venv`).
- Launch from the repository root: `python -m streamlit run app.py`.
- Reference input: first data row in `data/processed/validation.csv`.

Planned tests and results

PASS means the app or script behaved as expected. An error message can be a PASS when the test is checking whether the app rejects invalid input.

ID	Input or action	Expected behaviour	Actual result	Status
T01	Load the existing sample and run the prediction.	Show a class, a label, an educational notice and a CSV download option.	Showed class 1, `Predicted potable class`, and the educational notice. The CSV downloaded successfully.	PASS
T02	Reload the sample and set pH to `0`.	Accept the lower limit and return a prediction.	Accepted pH `0` and returned class 1.	PASS
T03	Reload the sample and set pH to `14`.	Accept the upper limit and return a prediction.	Accepted pH `14` and returned class 0.	PASS
T04	Set pH to `15` and submit the form.	Explain the allowed range and show no prediction.	Displayed `ph: use a value from 0 to 14.` No prediction was shown.	PASS
T05	Enter `one` in the pH field and submit the form.	Ask for a number and show no prediction.	Displayed `ph: enter a number.` No prediction was shown.	PASS
T06	Leave Trihalomethanes blank. Keep the other inputs valid, including pH `14`.	Name the missing field and show no prediction.	Displayed `Trihalomethanes: enter a value.` No prediction was shown.	PASS

Top of the preview: the project and environment, followed by the first six test rows.



on.csv U | STEP-6.md U | compare_step6_predictions.py U | COMMIT_EDITMSG | requirements-product-v1.txt | product_predict.py | app.py | check_product_prediction.py

docs > STEP-6.md — Markdown

Planned tests and results

PASS means the app or script behaved as expected. An error message can be a PASS when the test is checking whether the app rejects invalid input.

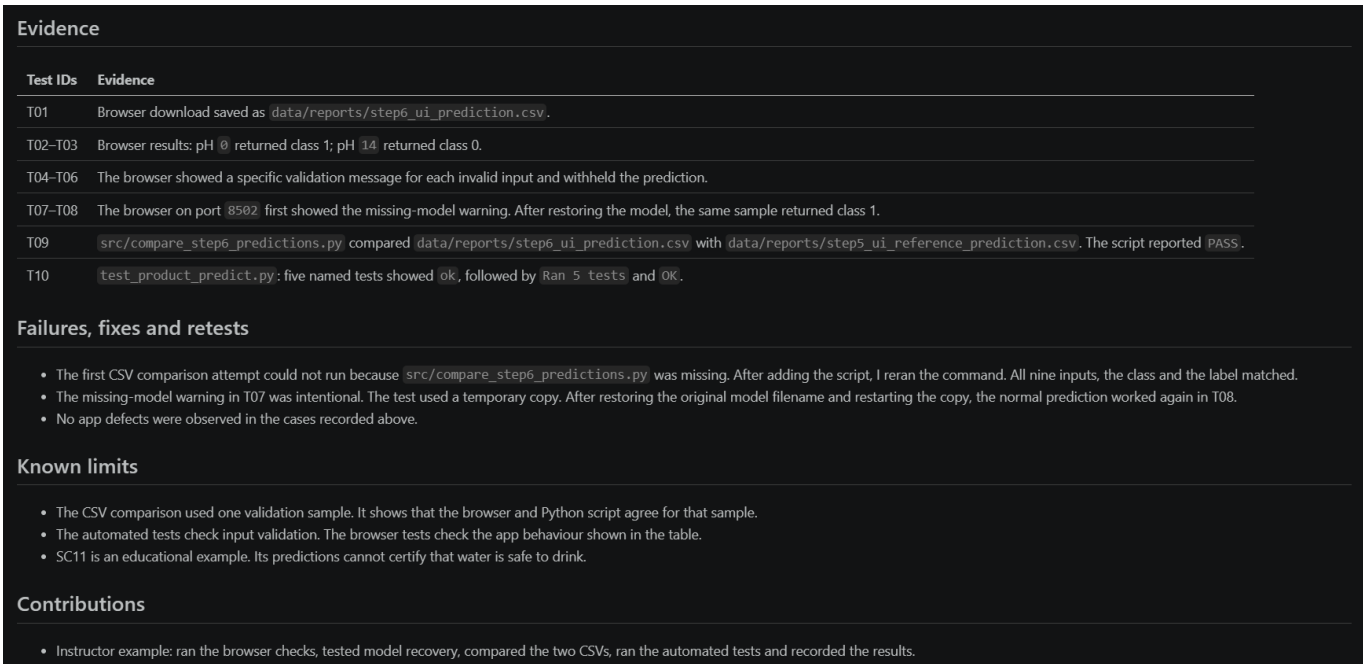
ID	Input or action	Expected behaviour	Actual result	Status
T01	Load the existing sample and run the prediction.	Show a class, a label, an educational notice and a CSV download option.	Showed class 1, `Predicted potable class`, and the educational notice. The CSV downloaded successfully.	PASS
T02	Reload the sample and set pH to `0`.	Accept the lower limit and return a prediction.	Accepted pH `0` and returned class 1.	PASS
T03	Reload the sample and set pH to `14`.	Accept the upper limit and return a prediction.	Accepted pH `14` and returned class 0.	PASS
T04	Set pH to `15` and submit the form.	Explain the allowed range and show no prediction.	Displayed `ph: use a value from 0 to 14.` No prediction was shown.	PASS
T05	Enter `one` in the pH field and submit the form.	Ask for a number and show no prediction.	Displayed `ph: enter a number.` No prediction was shown.	PASS
T06	Leave Trihalomethanes blank. Keep the other inputs valid, including pH `14`.	Name the missing field and show no prediction.	Displayed `Trihalomethanes: enter a value.` No prediction was shown.	PASS
T07	Rename the saved model in a temporary copy and start that copy on port `8502`.	Explain that the model or metadata is missing and hide the prediction form.	The browser showed `Saved model or metadata is missing. Restore your Step 3 files.` The prediction form was unavailable.	PASS
T08	Restore the model filename, restart the copy and predict using the existing sample.	Show the form again and return a prediction for the same sample.	The form returned and showed `Predicted potable class` and `Model class: 1`.	PASS
T09	Run `python src/compare_step6_predictions.py` to compare the browser and Python CSVs.	All nine measurements, the predicted class and the displayed label should match.	The script reported `PASS`. All nine measurements matched. Both results showed class 1 and `Predicted potable class`.	PASS
T10	Run `python -m unittest -v test_product_predict`.	All five input validation tests should pass.	All five tests showed `ok`. The terminal reported `Ran 5 tests` and ended with `OK`.	PASS

Scroll down to see T07 through T10. T07 passed because the missing-model warning was expected. T10 records that all five automated tests passed.

Action 9: record evidence, retests and limits

A result tells the reviewer what happened. Evidence helps the reviewer find or reproduce it.

- 4. Under Evidence, add a file path from your project or the exact result you observed. SC11 lists the browser CSV and records the test output.
- 5. Under Failures, fixes and retests, say what failed, what you changed and what happened when you repeated the check. If you found no defect, say so. Under Known limits, state what the tests did not cover.
- What happened in SC11. The CSV comparison could not run until its script was added. The next run showed that all nine inputs, the class and the label matched.
- In your project, record your own results and each team member's contribution. Save and check that the finished report is readable in Preview.



Evidence

Test IDs	Evidence
T01	Browser download saved as `data/reports/step6_ui_prediction.csv`.
T02–T03	Browser results: pH `0` returned class 1; pH `14` returned class 0.
T04–T06	The browser showed a specific validation message for each invalid input and withheld the prediction.
T07–T08	The browser on port `8502` first showed the missing-model warning. After restoring the model, the same sample returned class 1.
T09	`src/compare_step6_predictions.py` compared `data/reports/step6_ui_prediction.csv` with `data/reports/step5_ui_reference_prediction.csv`. The script reported `PASS`.
T10	`test_product_predict.py`: five named tests showed `ok`, followed by `Ran 5 tests` and `OK`.

Failures, fixes and retests

- The first CSV comparison attempt could not run because `src/compare_step6_predictions.py` was missing. After adding the script, I reran the command. All nine inputs, the class and the label matched.
- The missing-model warning in T07 was intentional. The test used a temporary copy. After restoring the original model filename and restarting the copy, the normal prediction worked again in T08.
- No app defects were observed in the cases recorded above.

Known limits

- The CSV comparison used one validation sample. It shows that the browser and Python script agree for that sample.
- The automated tests check input validation. The browser tests check the app behaviour shown in the table.
- SC11 is an educational example. Its predictions cannot certify that water is safe to drink.

Contributions

- Instructor example: ran the browser checks, tested model recovery, compared the two CSVs, ran the automated tests and recorded the results.

SC11's evidence, retests, known limits and contributions in the same report.

Action 10: stage and commit the Step 6 files

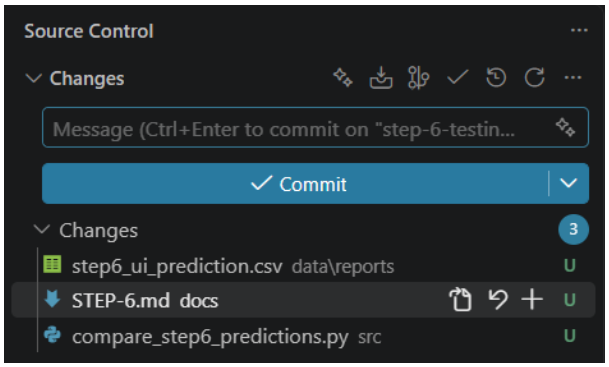
Staging selects the files for the commit. A commit saves those files in your local Git history. The next page shows how to send it to GitHub.

- 1. Save your files with Ctrl+S. Open Source Control from the left sidebar (Ctrl+Shift+G). Check the bottom-left branch name. In SC11 it should say step-6-testing.
- 2. Under Changes, click a file to review it. Hover over the filename and click its + button to move it to Staged Changes. Stage only the files that belong in this Step 6 commit.

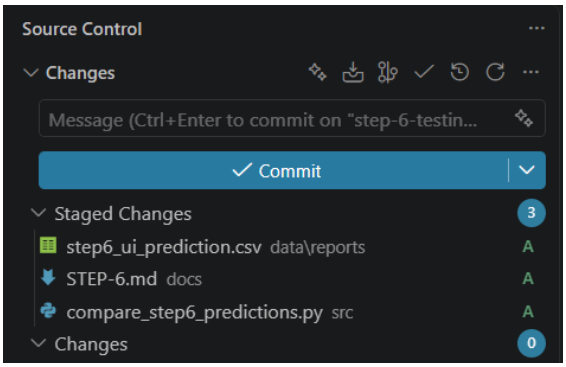
SC11 files to look for under Changes

```
docs/STEP-6.md
src/compare_step6_predictions.py
data/reports/step6_ui_prediction.csv
```

- In your project, stage the report, changed test code and shareable evidence. Leave out .venv, caches, credentials and unrelated files.
- 3. Check the list under Staged Changes. In the message box, describe what you saved, for example Add SC11 Step 6 test evidence. Click Commit.
- What to expect. The files you committed disappear from Staged Changes. A local commit can exist even if GitHub still shows the old version.



Under Changes: click the + beside each file.

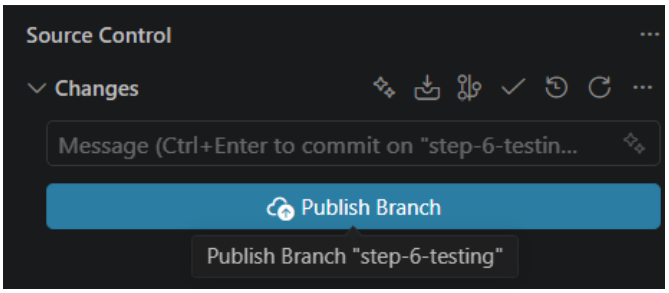


Under Staged Changes: confirm the three files, then commit.

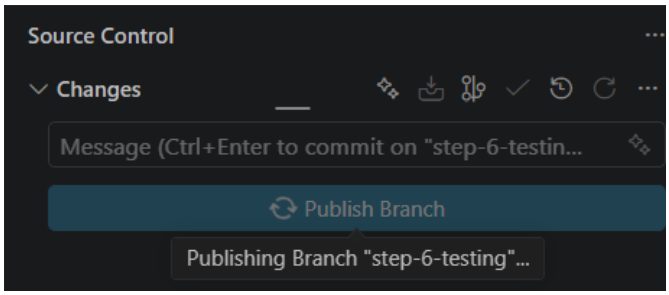
Action 10: publish or push the branch

A commit is local until you send it to GitHub. Publishing sends a branch for the first time; pushing sends new commits to a branch that already exists there.

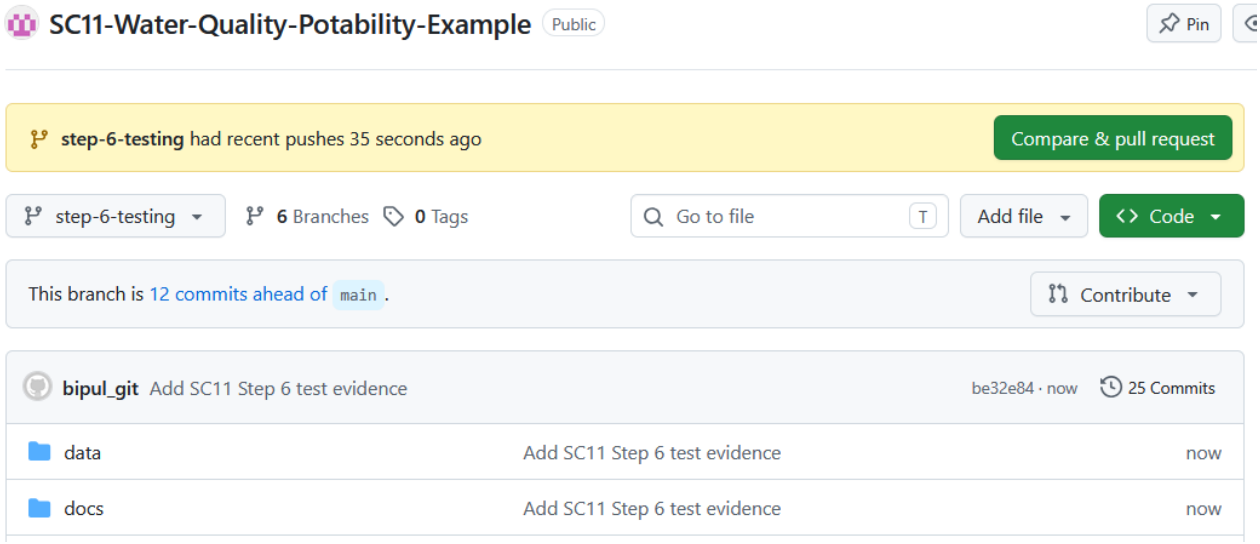
- 4. In Source Control, click Publish Branch if this branch has never been pushed. If it already exists on GitHub, use the three-dot menu > Push. If prompted, choose the remote for your existing team repository.
- What happened in SC11. VS Code showed Publishing Branch while the upload ran. Wait for it to finish before checking GitHub.



Before the first push: click Publish Branch.



While VS Code publishes: wait for it to finish.



On GitHub, SC11 shows step-6-testing, the new commit and the docs folder.

Action 10: verify the report on GitHub

Opening the report on GitHub confirms that the reviewer can see the work you just committed.

- 5. Open your GitHub repository. Use the branch selector, which may say main, and choose your Step 6 branch. Check that the latest commit matches what you entered in VS Code.
- 6. On that branch, open docs > STEP-6.md. Check the results table and make sure that the files listed as evidence are available in the same branch.
- What happened in SC11. The published step-6-testing branch showed the new commit and the report opened on GitHub. Use your own public repository when you submit in Action 11.

Step 6 - Testing and validation

Project and environment

- Project: SC11 Water Quality Potability Example (instructor reference).
- Branch: `step-6-testing`.
- Commit tested: `e483875`.
- Python version: `3.12.1` (project `.venv`).
- Launch from the repository root: `python -m streamlit run app.py`.
- Reference input: first data row in `data/processed/validation.csv`.

Planned tests and results

PASS means the app or script behaved as expected. An error message can be a PASS when the test is checking whether the app rejects invalid input.

ID	Input or action	Expected behaviour	Actual result	Status
T01	Load the existing sample and run the prediction.	Show a class, a label, an educational notice and a CSV download option.	Showed class 1, `Predicted potable class`, and the educational notice. The CSV downloaded successfully.	PASS
T02	Reload the sample and set pH to `0`.	Accept the lower limit and return a prediction.	Accepted pH `0` and returned class 1.	PASS
T03	Reload the sample and set pH to `14`.	Accept the upper limit and return a prediction.	Accepted pH `14` and returned class 0.	PASS

The Step 6 report rendered on GitHub, with the tested environment and results table.

Action 11: Submit Step 6 in the project portal

The completion note tells the reviewer where to find your test evidence.

- What to do. Sign in to your team dashboard and choose Step 6 after submitting Step 5. Enter your team's public GitHub repository URL.
- The public repository URL is required. Use the Brief completion note (up to 700 characters) to give the branch, report path, test command or file, and a short result or open issue. Select Submit work.
- What to expect. The portal shows a confirmation and updates the Step 6 status.
- In your project, verify the confirmation for your team. The deadline is 22 November 2026 at 11:59 PM IST.

Example completion note

Branch step-6-testing. Report docs/STEP-6.md. Automated check python -m unittest -v test_product_predict. See the report for actual results and remaining issues.

- Open the GitHub branch and check that the report loads.
- Each planned case has a result or is marked NOT RUN.
- SC11 is the instructor example. Allocated teams submit their own repositories.